

ECE 315

Steven Knudsen, PhD, PEng

Lecture 21:

- RTM
- Memory and memory-mapped I/O



UNIVERSITY
OF ALBERTA

Last Lecture

- Computer Interfaces, in general
- Planning peripherals and interfaces
- Design and build for test

This Lecture

- RTM (refer to the manual)
- Memory and memory-mapped I/O

Learning Objectives

- Practice “Refer to the Manual” (RTM) as a critical engineering habit
- Differentiate between major types of memory-mapped systems
- Explain the concept and purpose of virtual memory
- Describe how file-backed memory mapping works
- Describe how anonymous memory mapping works
- Compare memory-mapped allocation to ordinary virtual memory allocation
- Understand and analyze an example program involving file-mapped/backed memory, and how it might be made anonymous

RTM

Refer to The Manual (RTM)

- Always, always read the documentation
 - Data sheets,
 - Errata,
 - Application notes,
 - Application Programming Interface (API) docs
 - Manuals,
 - Tutorials,
 - Examples
- Be sure to check the change/update logs

Refer to The Manual (RTM)

- Always, always read the documentation
 - Data sheets,
 - Errata,
 - Application notes,
 - Application Programming Interface (API) docs
 - Manuals,
 - Tutorials,
 - Examples
- Be sure to check the change/update logs



MPU-6050
3-axis MEMS
accelerometer and
gyroscope



7

MPU-6050 RTM

Main data sheet

- Title page already tells us the revision
- Checking the data sheet we originally used, we see we are out of date
- We used version 3.0



	InvenSense Inc. 1197 Barragans Ave, Sunnyvale, CA 94089 U.S.A. Tel: +1 (408) 598-7239 Fax: +1 (408) 598-8104 Website: www.invensense.com	Document Number: PS-MPU-6000A-00 Revision: 3.4 Release Date: 08/19/2013
---	---	--

MPU-6000 and MPU-6050 Product Specification Revision 3.4

1 of 52

MPU-6050 RTM

Main data sheet

- Revision history contains some pretty important changes



	MPU-6000/MPU-6050 Product Specification	Document Number: PS-MPU-6000A-00 Revision: 3.4 Release Date: 08/19/2013
---	---	---

1 Revision History

Revision Date	Revision	Description
11/24/2010	1.0	Initial Release
05/19/2011	2.0	For Rev C parts. Clarified wording in sections (3.2, 5.1, 5.2, 6.1-6.4, 6.6, 6.9, 7, 7.1-7.6, 7.11, 7.12, 7.14, 8, 8.2-8.4, 10.3, 10.4, 11, 12.2)
07/28/2011	2.1	Edited supply current numbers for different modes (section 6.4)
08/05/2011	2.2	Unit of measure for accelerometer sensitivity changed from LSB/mg to LSB/g
10/12/2011	2.3	Updated accelerometer self test specifications in Table 6.2. Updated package dimensions (section 11.2). Updated PCB design guidelines (section 11.3)
10/18/2011	3.0	For Rev D parts. Updated accelerometer specifications in Table 6.2. Updated accelerometer specification note (sections 8.2, 8.3, & 8.4). Updated qualification test plan (section 12.2).
10/24/2011	3.1	Edits for clarity Changed operating voltage range to 2.375V-3.46V Added accelerometer Intelligence Function increment value of 1mg/LSB (Section 6.2) Updated absolute maximum rating for acceleration (any axis, unpowered) from 0.3ms to 0.2ms (Section 6.9) Modified absolute maximum rating for Latch-up to Level A and $\pm 100\text{mA}$ (Section 6.9, 12.2)
11/16/2011	3.2	Updated self-test response specifications for Revision D parts dated with date code 1147 (YYWW) or later. Edits for clarity Added Gyro self-test (sections 5.1, 6.1, 7.6, 7.12) Added Min/Max limits to Accel self-test response (section 6.2) Updated Accelerometer low power mode operating currents (Section 6.3) Added gyro self test to block diagram (section 7.5) Updated packaging labels and descriptions (sections 11.8 & 11.9)
5/16/2012	3.3	Updated Gyro and Accelerometer self test information (sections 6.1, 6.2, 7.12) Updated latch-up information (Section 6.9) Updated programmable interrupts information (Section 8) Changed shipment information from maximum of 3 reels (15K units) per shipper box to 5 reels (25K units) per shipper box (Section 11.7) Updated packing shipping and label information (Sections 11.8, 11.9) Updated reliability references (Section 12.2)
8/19/2013	3.4	Updates section 4

9

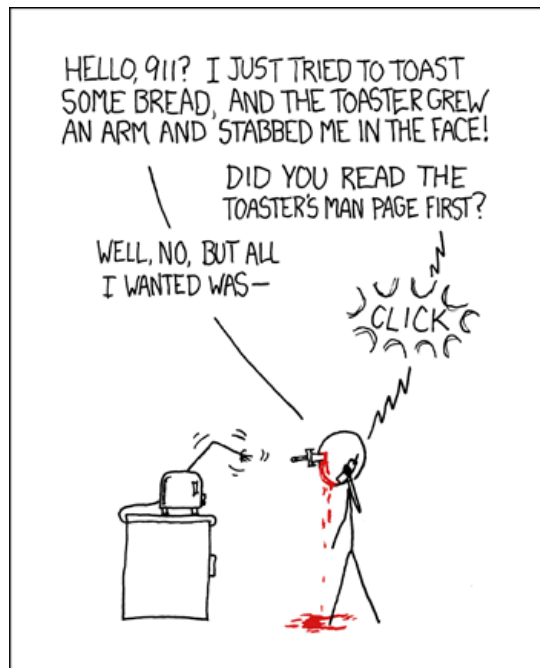
MPU-6050 RTM

Main data sheet

- Every revision has items that we should really pay attention to(!)



10/18/2011	3.0	For Rev D parts. Updated accelerometer specifications in Table 6.2. Updated accelerometer specification note (sections 8.2, 8.3, & 8.4). Updated qualification test plan (section 12.2).
10/24/2011	3.1	Edits for clarity Changed operating voltage range to 2.375V-3.46V Added accelerometer Intelligence Function increment value of 1mg/LSB (Section 6.2) Updated absolute maximum rating for acceleration (any axis, unpowered) from 0.3ms to 0.2ms (Section 6.9) Modified absolute maximum rating for Latch-up to Level A and $\pm 100\text{mA}$ (Section 6.9, 12.2)
11/16/2011	3.2	Updated self-test response specifications for Revision D parts dated with date code 1147 (YYWW) or later. Edits for clarity Added Gyro self-test (sections 5.1, 6.1, 7.6, 7.12) Added Min/Max limits to Accel self-test response (section 6.2) Updated Accelerometer low power mode operating currents (Section 6.3) Added gyro self test to block diagram (section 7.5) Updated packaging labels and descriptions (sections 11.8 & 11.9)
5/16/2012	3.3	Updated Gyro and Accelerometer self test information (sections 6.1, 6.2, 7.12) Updated latch-up information (Section 6.9) Updated programmable interrupts information (Section 8) Changed shipment information from maximum of 3 reels (15K units) per shipper box to 5 reels (25K units) per shipper box (Section 11.7) Updated packing shipping and label information (Sections 11.8, 11.9) Updated reliability references (Section 12.2)
8/19/2013	3.4	Updates section 4



Memory-Mapping

Memory-mapped Systems

There are three kinds of memory-mapped systems of general interest

- **File Mapping**
 - Make a file appear to be all in virtual memory
 - Read-only (RO) or Read-Write (RW)
- **Anonymous Mapping**
 - Supports extended “RAM” storage for operating systems
 - Makes the system appear to have more RAM than it physically has
- **Device mapping**
 - Treat peripherals as addressable entities

Memory-mapped Systems – Programmer's perspective

There are three kinds of memory-mapped systems of general interest

- File Mapping – **OSes; Linux, Windows, OSX, etc.**
 - Make a file appear to be all in virtual memory
 - Read-only (RO) or Read-Write (RW)
- Anonymous Mapping – **OSes; Linux, Windows, OSX, etc.**
 - Supports extended “RAM” storage for operating systems
 - Makes the system appear to have more RAM than it physically has
- Device mapping – **Embedded, Usually**
 - Treat peripherals as addressable entities



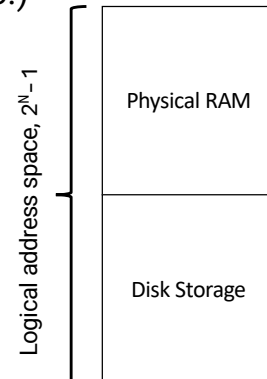
Based on <https://www.geeksforgeeks.org/operating-systems/memory-mapping/>

14

In this lecture we look at the first two, File and Anonymous mapping.

Memory Mapping

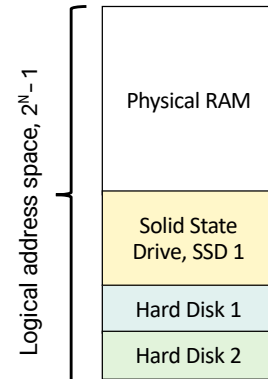
- Modern computers (mainframes, laptops, desktops, etc.) have a virtual memory that is not physically present
- Consider an N-bit processor
 - If $N=32$, then $2^{32} - 1 = 4\text{GB}$, which could easily be all RAM
 - If $N=64$, then $2^{64} - 1 = 17,179,869,164\text{GB}$; no way that can be all RAM
- Instead, we can extend the available RAM “virtually” with another form of storage, often a disk (HD, SSD)
- A `mmap()` function or device will “map” the extended space to bytes on the secondary (disk) storage



File Memory Mapping

File Mapping – file is allocated from *virtual memory*

- Can be set up as read-only (RO) or read-write (RW)
- Writing to RO will cause an operating system error
 - A good programmer will handle it gracefully; no crash
- The mapped storage is handled as a file
 - All the usual file operations can be done with it
 - Logically, starts at an address and is contiguous
 - The physical storage it's mapped onto may or may not be contiguous
 - The memory mapping function, `mmap ( )`, manages that abstraction



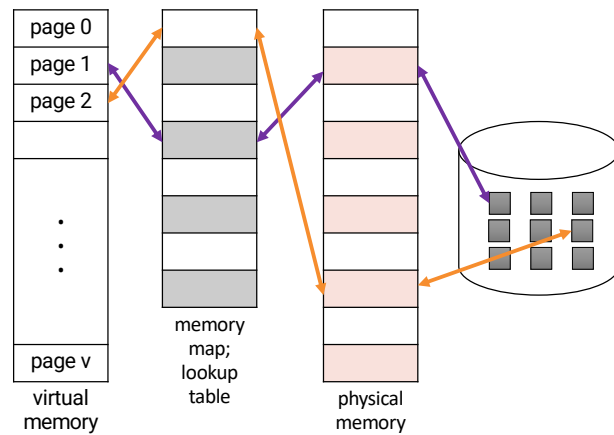
Based on <https://www.geeksforgeeks.org/operating-systems/memory-mapping/>

16

We could have a logical view like this, but there are many other organizations possible.

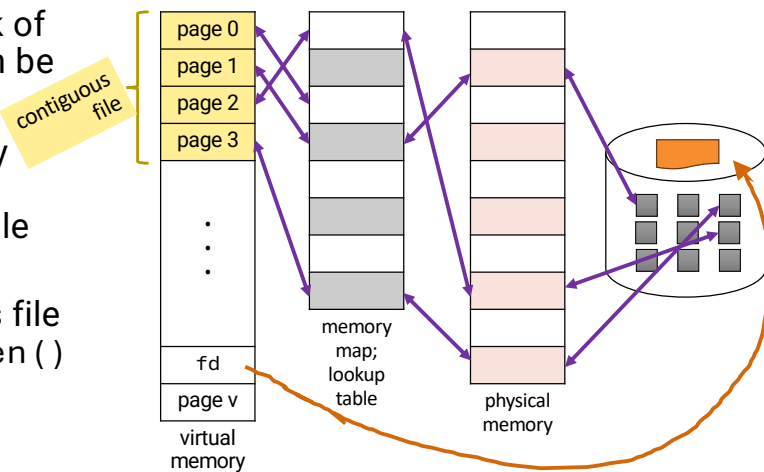
Aside – Virtual Memory

- Gives the appearance of a large block of memory even if the physical RAM is much smaller
- Main idea
 - Divide the process memory into contiguous “pages” of fixed size
 - They are in “virtual memory”
 - Store the pages on disk and when the program needs the memory mapped to a specific page, fetch it and place in RAM



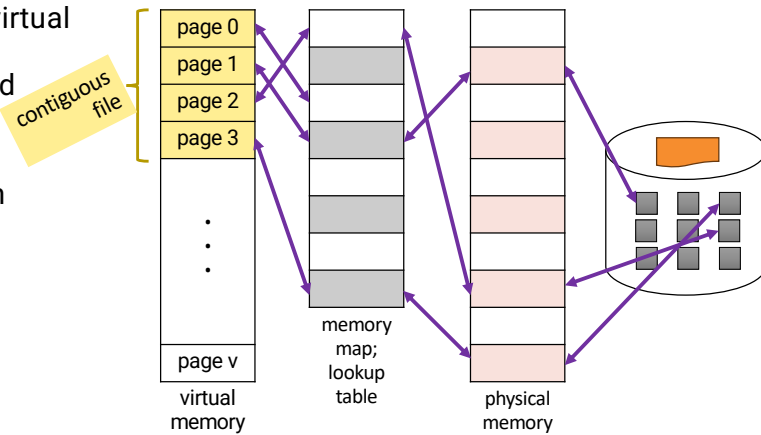
File Memory Mapping

- `mmap ( )` returns a block of virtual memory that can be treated as a file
- The “file” is backed by physical file on disk of pages pointed to by a file descriptor (`fd`)
- Use the `fd` with various file functions including `open ( )` and `close ( )`

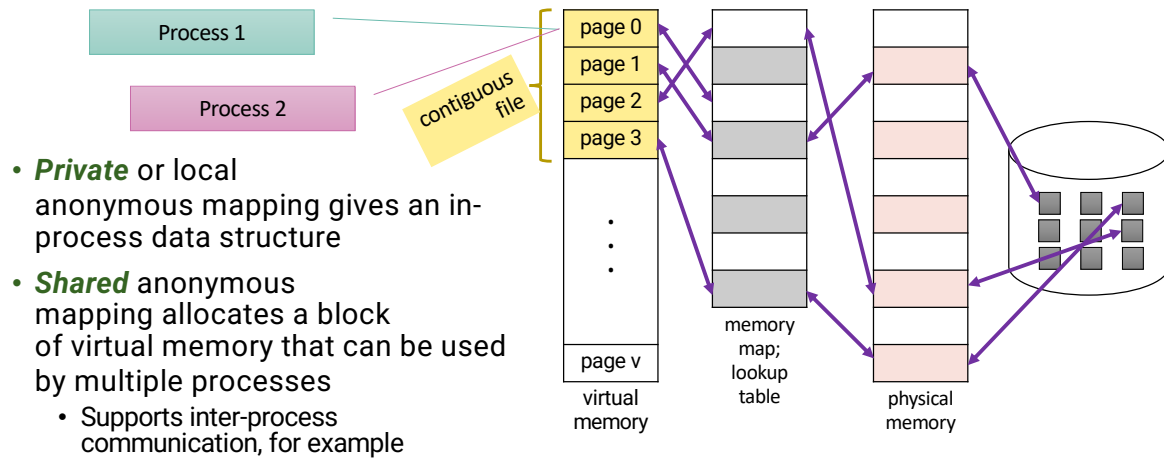


Anonymous Memory Mapping

- `mmap ( )` returns a block of virtual memory that can be treated as file, but unbacked by a physical file
- Can be local to the calling process, or shared between processes

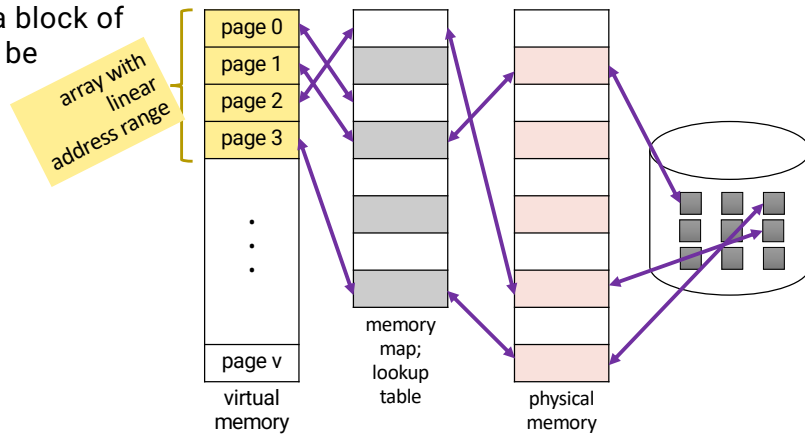


Anonymous and File Memory Mapping



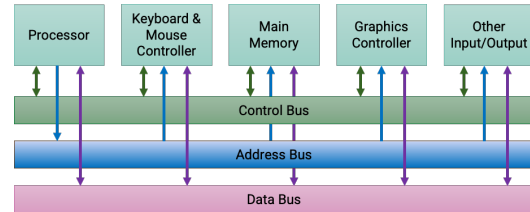
Plain Old Virtual Memory

- E.g., `malloc()` returns a block of virtual memory that can be treated as a linear address array
- Or static allocation
- Or `new()`, ...



What does this have to do with interfacing?

- How the computer bus is set up will impact how the virtual memory function works
 - Both logically and physically
- The width (number of signal lines) dictates the addressable physical memory
- However, a disk may be only one or a small range of addresses on the physical bus
- The memory mapper, whether implemented in software or as hardware manages the mapping



File Mapping – Example program

General approach

1. Start
2. Initialize the size of the file and mapping.
3. If the file not exists, create file; Open the file.
4. If exists, truncate the file to the desired size.
5. Create a memory-mapped region.
6. Initialize memory-mapped region with some data.
7. Read and print the contents of the memory mapped region.
8. Un-map the memory and close the file.
9. Stop



Based on <https://www.geeksforgeeks.org/operating-systems/memory-mapping/>

23

File Mapping – Example program

- A default path is assumed, in this case where the program executable is on disk
- The file is created if it does not already exist
- Force to be length file_size via ftruncate()

Based on <https://www.geeksforgeeks.org/operating-systems/memory-mapping/>



UNIVERSITY
OF ALBERTA

24

```
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <unistd.h>

int main()
{
    const char* file_path = "GFG-M-Mapping.txt";
    const size_t file_size
        = 4096; // Initializing file size to $kb.

    // We open the file for read/write operation
    int fd = open(
        file_path, O_RDWR | O_CREAT,
        0400 | 0200); // Use octal values for permissions
    if (fd == -1) {
        perror("open");
        return 1;
    }

    // if already exist, we truncate it to desired size
    if (ftruncate(fd, file_size) == -1) {
        perror("ftruncate");
        close(fd);
        return 1;
    }

    // Create a memory-mapped region for the file.
    void* file_memory
        = mmap(NULL, file_size, PROT_READ | PROT_WRITE,
            MAP_SHARED, fd, 0);
    if (file_memory == MAP_FAILED) {
        perror("mmap");
        close(fd);
        return 1;
    }

    // Initialize the memory-mapped region with some data.
    const char* message
        = "Hello Geeks, Memory mapping is done !";
    strncpy(file_memory, message, strlen(message));

    // Read and print the contents of the memory-mapped
    // region.
    printf("Contents of the memory-mapped region: %s\n",
        (char*)file_memory);

    // Unmap the memory and close the file.
    munmap(file_memory, file_size);
    close(fd);

    return 0;
}
```

File Mapping – Example program

- Use `mmap()` to create a memory-mapped region in virtual memory
- If the address is `NULL`, the OS kernel chooses a memory location
- `MAP_SHARED` makes this file-mapped so changes to the shared file memory are written back to the physical file
- If the address was specified, other processes can share the file in virtual memory
- Could have chosen `MAP_PRIVATE`, then other processes can't see the file

Based on <https://www.geeksforgeeks.org/operating-systems/memory-mapping/>



25

```
// Create a memory-mapped region for the file.
void* file_memory
    = mmap(NULL, file_size, PROT_READ | PROT_WRITE,
           MAP_SHARED, fd, 0);
if (file_memory == MAP_FAILED) {
    perror("mmap");
    close(fd);
    return 1;
}

// Initialize the memory-mapped region with some data.
const char* message
    = "Hello Geeks,Memory mapping is done !";
strncpy(file_memory, message, strlen(message));

// Read and print the contents of the memory-mapped
// region.
printf("Contents of the memory-mapped region: %s\n",
       (char*)file_memory);

// Unmap the memory and close the file.
munmap(file_memory, file_size);
close(fd);

return 0;
}
```

Anonymous Mapping – Example program

- Specifying MAP_ANONYMOUS instead of MAP_SHARED results in virtual memory-only file, fd is ignored and there is no matching on-disk file
- The allocated file contents are initialized to zero (0)
- When munmap () is called, all the changes are lost

Based on <https://www.geeksforgeeks.org/operating-systems/memory-mapping/>



26

```
// Create a memory-mapped region for the file.
void* file_memory
    = mmap(NULL, file_size, PROT_READ | PROT_WRITE,
           MAP_SHARED, fd, 0);
if (file_memory == MAP_FAILED) {
    perror("mmap");
    close(fd);
    return 1;
}

// Initialize the memory-mapped region with some data.
const char* message
    = "Hello Geeks,Memory mapping is done !";
strncpy(file_memory, message, strlen(message));

// Read and print the contents of the memory-mapped
// region.
printf("Contents of the memory-mapped region: %s\n",
       (char*)file_memory);

// Unmap the memory and close the file.
munmap(file_memory, file_size);
close(fd);

return 0;
}
```

Memory Mapping Summary

- Memory mapping with the support of virtual memory provides a nice logical perspective on the memory space of the system
- Helps make programs more portable as you don't have to know the actual physical address of a memory object
- "Abstracts away" the underlying hardware
- File mapping allows file data to be manipulated in RAM (virtual memory) which is fast, and when/if needed the file can be saved to disk
- Also allows sharing of file structures between processes

Next Lecture

- Memory mapping to peripherals

Questions

